RÉPUBLIQUE DU CAMEROUN

Paix · Travail · Patrie

UNIVERSITÉ DE YAOUNDÉ I

ÉCOLE NATIONALE SUPÉRIEURE POLYTECHNIQUE

DÉPARTEMENT DE GÉNIE INFORMATIQUE

REPUBLIC OF CAMEROON

Peace · Work · Fatherland

UNIVERSITY OF YAOUNDÉ I

NATIONAL ADVANCED SCHOOL OF ENGINEERING

DEPARTMENT OF COMPUTER ENGINEERING

RAPPORT DE STAGE

Stage au sein du Laboratoire IoT

Internet des Objets et Systèmes Embarqués

Stagiaires NOUMSI Elvira
NJOME David
TAGNE Lavoisier
TCHOUNOU Maeva
MEGNIJIO Lea

Encadreur Dr Anne Marie Chana

Année académique 2025 – 2026

INFORMATIONS SUR LE STAGE

Structure d'accueil	Laboratoire IoT de l'École Nationale Supérieure Polytechnique de Yaoundé
Domaine d'activité	Internet des Objets (IoT) et électronique embarquée
Travaux réalisés	Étude du kit Keyestudio Arduino et participation au projet Eyeshome
Période du stage	8 juillet 2026 – 4 septembre 2026
Filière académique	Génie informatique

Remerciements

Le présent rapport constitue l'aboutissement de notre stage effectué au sein du Laboratoire IoT de l'École Nationale Supérieure Polytechnique de Yaoundé. Cette expérience nous a permis de mettre en pratique les connaissances acquises au cours de notre formation et de développer de nouvelles compétences dans le domaine de l'Internet des Objets et des systèmes embarqués, plus précisément avec RIOT et LoRa.

Nous tenons tout d'abord à exprimer notre profonde gratitude à l'École Nationale Supérieure Polytechnique de Yaoundé pour la formation reçue et pour les opportunités d'apprentissage et d'expérimentation qu'elle offre à ses étudiants.

Nous adressons également nos sincères remerciements à l'ensemble des enseignants et à nos aînés du Laboratoire IoT pour leur accueil, leur disponibilité et les conditions de travail mises à notre disposition durant notre stage.

Nous exprimons particulièrement notre reconnaissance au **Dr Anne Marie Chana**, notre encadreur, pour son accompagnement, ses orientations, ses conseils et le suivi accordé à nos travaux tout au long du stage.

Enfin, nous remercions également toutes les personnes qui ont contribué, directement ou indirectement, au bon déroulement de cette expérience, notamment à travers leurs conseils, leurs remarques et leurs encouragements.

À toutes et à tous, nous adressons nos sincères remerciements.

Résumé

Dans le cadre de notre formation en informatique, nous avons effectué un stage au sein du Laboratoire IoT de l'École Nationale Supérieure Polytechnique de Yaoundé. Ce stage avait pour objectif de nous familiariser avec l'environnement de l'Internet des Objets, les systèmes embarqués ainsi qu'avec les méthodes de travail et d'expérimentation mises en œuvre dans un environnement de laboratoire.

La première phase du stage a été consacrée à la découverte et à l'étude de l'électronique à travers le kit **Keyestudio Arduino**. Cette étape nous a permis de prendre en main différents composants électroniques, de comprendre leur fonctionnement et de réaliser plusieurs expérimentations permettant de mettre en pratique les notions étudiées.

La seconde phase a porté sur notre participation au projet **Eyeshome**, une solution orientée vers la surveillance d'un environnement domestique via un petit dispositif. Le système permet notamment de suivre différents paramètres tels que la présence, la température, l'humidité ainsi que le niveau de gaz dans une pièce, avec une actualisation des données sur une application lorsque les valeurs évoluent.

L'ensemble de ces travaux a été réalisé dans un cadre collaboratif, ce qui nous a également permis de développer des compétences en communication, en organisation, en recherche documentaire, en résolution de problèmes et en travail en équipe.

Ce stage a ainsi constitué une expérience complémentaire à notre formation, en nous permettant de confronter nos connaissances théoriques à des situations pratiques et de mieux comprendre les différentes étapes intervenant dans la réalisation d'une solution basée sur les technologies IoT.

Mots-clés : Internet des Objets (IoT), Arduino, Keyestudio, systèmes embarqués, capteurs, RIOT OS, LoRaWAN, travail en équipe.

Abstract

As part of our computer engineering training, we carried out an internship at the IoT Laboratory of the National Advanced School of Engineering of Yaoundé. The main objective of this internship was to familiarize ourselves with the Internet of Things, embedded systems, as well as the working and experimentation methods used in a laboratory environment.

The first phase of the internship focused on discovering and studying electronics through the **Keyestudio Arduino kit**. This phase enabled us to become familiar with several electronic components, understand their operating principles and carry out practical experiments to apply the knowledge acquired during our training.

The second phase involved our participation in the **Eyeshome** project, a solution designed for monitoring a domestic environment via a small device. The system is intended to monitor several parameters, including presence, temperature, humidity and the gas level in a room, with data being updated on an application whenever the measured values change.

The activities were carried out within a collaborative framework, allowing us to develop skills in communication, organization, technical research, problem solving and teamwork.

This internship therefore provided valuable practical experience complementary to our academic training. It enabled us to confront theoretical knowledge with practical situations and to gain a better understanding of the different stages involved in the development of an IoT-based solution.

Keywords : Internet of Things (IoT), Arduino, Keyestudio, embedded systems, sensors, RIOT OS, LoRaWAN, teamwork.

Table des matières

Introduction générale **11**

I Cadre général du stage **12**

1 Présentation de la structure d'accueil **13**

2 Organisation et déroulement du stage **14**

II Concepts techniques fondamentaux **15**

3 Microcontrôleurs et systèmes embarqués **16**
3.1 Qu'est-ce qu'un microcontrôleur ? . . . 16
3.2 Fonctionnement général d'un microcontrôleur . . . 17
3.3 Architecture d'un microcontrôleur . . . 17
3.3.1 Unité centrale de traitement (CPU) . . . 17
3.3.2 Mémoire . . . 17
3.3.3 Unités d'entrée/sortie (GPIO) . . . 17
3.3.4 Interfaces de communication . . . 18
3.3.5 Horloge . . . 18
3.4 Exemples de microcontrôleurs courants . . . 18
3.5 Lien entre microcontrôleurs et l'IoT . . . 19

4 RIOT OS : Système d'exploitation pour l'IoT **21**
4.1 Introduction à RIOT OS . . . 21
4.2 Caractéristiques principales . . . 21
4.3 Avantages de RIOT OS . . . 21
4.4 Architecture technique de RIOT OS . . . 22
4.5 Installation et compilation . . . 22
4.6 Applications et cas d'usage . . . 22

5 La carte Wyres Base **23**
5.1 Présentation générale . . . 23
5.2 Fonctions principales . . . 23
5.3 Caractéristiques techniques détaillées . . . 24
5.4 Connexion et programmation . . . 24
5.4.1 Identification du port série et vérification de la connexion avec la carte Wyres Base . . . 24
5.4.2 Branchement pour la programmation . . . 25
5.5 Applications courantes de la carte Wyres Base . . . 26

5.6 Avantages de la carte Wyres Base . . . 26
5.7 Limites de la carte Wyres Base . . . 27
5.8 Schéma fonctionnel simplifié du fonctionnement de la carte Wyres Base . . . 27

6 Technologies LoRa et LoRaWAN 29
6.1 LoRa : Technologie de modulation . . . 29
6.2 Comparaison avec d'autres technologies . . . 29
6.3 Quelques applications de LoRa . . . 30
6.4 LoRaWAN : Protocole réseau . . . 30
6.5 Architecture LoRaWAN . . . 30
6.6 Classes de dispositifs LoRaWAN . . . 31
6.7 Sécurité dans LoRaWAN . . . 31

III Déroulement et apprentissages du stage 32

7 Apprentissage pratique : Super Learning Kit for Arduino de Keyestudio (KS0077/KS0078/KS0079)
7.1 Présentation du kit et composants . . . 33
7.2 Étude des composants fondamentaux . . . 34
7.3 Expérimentations réalisées . . . 34

8 Participation au projet Eyeshome 37
8.1 Présentation du projet . . . 37
8.2 Organisation du travail en équipe . . . 37

IV Bilan et perspectives 40

9 Compétences acquises 41
9.1 Compétences techniques . . . 41
9.2 Compétences méthodologiques . . . 41
9.3 Compétences professionnelles . . . 41

10 Difficultés rencontrées et solutions apportées 42
10.1 Difficultés techniques . . . 42
10.2 Difficultés d'apprentissage . . . 42
10.3 Difficultés de travail en équipe . . . 42
10.4 Enseignements tirés . . . 42

11 Bilan critique et perspectives 43
11.1 Bilan global du stage . . . 43
11.2 Analyse critique . . . 43
11.3 Contribution au laboratoire . . . 43
11.4 Perspectives et évolutions futures . . . 43

Table des figures

3.1 Microcontrôleur de la marque Microchip Technology 16
3.2 Schéma fonctionnel d'un microcontrôleur 18

5.1 La carte Wyres Base 23
5.2 Schéma fonctionnel simplifié de la carte Wyres Base dans un projet IoT 28

6.1 Comparaison entre technologies 29
6.2 Structure globale d'un réseau LoRaWAN 30

7.1 Kit Keyestudio KS0077/KS0078/KS0079 33
7.2 Composants du kit Keyestudio KS0077/KS0078/KS0079 34
7.3 Montage du projet PWM : variation de la luminosité d'une LED à l'aide d'un potentiomètre . 35
7.4 Affichage d'un message sur l'écran LCD 1602 36

8.1 Architecture suivie pour le prototype avec WiFi 38
8.2 Architecture suivie pour le prototype avec LoRa/LoRaWAN 38

Liste des tableaux

3.1 Tableau comparatif des microcontrôleurs additionnels 19

5.1 Correspondance des broches entre le programmateur ST-Link et la carte Wyres Base 25
5.2 Correspondance des broches entre ST-Link Nucleo et la carte Wyres Base 26

Liste des sigles et abréviations

IoT	Internet of Things (Internet des Objets)
ENSPY	École Nationale Supérieure Polytechnique de Yaoundé
IDE	Integrated Development Environment
LED	Light Emitting Diode
RIOT OS	Real-time IoT Operating System
LoRa	Long Range
LoRaWAN	Long Range Wide Area Network
STM32	STMicroelectronics Microcontroller 32 bits
UART	Universal Asynchronous Receiver/Transmitter
I2C	Inter-Integrated Circuit
SPI	Serial Peripheral Interface

Introduction générale

L'évolution rapide des technologies numériques transforme progressivement les méthodes de conception, de communication et de traitement de l'information. Parmi les domaines qui illustrent particulièrement cette évolution figure l'Internet des Objets (IoT), qui permet à des objets physiques équipés de capteurs, de dispositifs de traitement et de moyens de communication de collecter, traiter et échanger des données.

Dans le cadre d'une formation en informatique, l'acquisition de connaissances théoriques doit être complétée par des expériences pratiques permettant de comprendre la mise en œuvre concrète des technologies étudiées. Le stage constitue ainsi une étape importante de la formation, puisqu'il permet de se confronter à un environnement de travail réel, d'appliquer ses connaissances et de développer de nouvelles compétences.

C'est dans cette perspective que nous avons effectué notre stage au sein du **Laboratoire IoT de l'École Nationale Supérieure Polytechnique de Yaoundé**. Ce cadre nous a permis d'évoluer dans un environnement orienté vers l'expérimentation et la réalisation de solutions reposant notamment sur l'Internet des Objets, les systèmes embarqués et leurs composants électroniques.

Le stage s'est déroulé en plusieurs étapes. Une première phase a été consacrée à l'étude et à la prise en main du **kit Keyestudio Arduino**. Cette étape nous a permis de découvrir différents composants, d'étudier leur fonctionnement et de réaliser des expérimentations pratiques.

Nous avons ensuite participé, dans le cadre d'un travail collectif, au projet **Eyeshome**. Ce projet porte sur la surveillance de plusieurs paramètres d'un environnement domestique, notamment la présence, la température, l'humidité et le niveau de gaz dans l'air. Les informations collectées sont transmises et actualisées en temps réel à une application afin de permettre leur suivi.

Au-delà des aspects techniques, ce stage nous a également permis de développer des compétences méthodologiques et professionnelles. Le travail en équipe, la recherche documentaire, l'expérimentation, l'analyse des difficultés et la recherche de solutions ont constitué des éléments importants de notre démarche.

Le présent rapport a pour objectif de présenter le cadre dans lequel le stage a été effectué, son déroulement, les principales activités réalisées ainsi que les connaissances et compétences acquises. Il ne constitue pas une présentation technique exhaustive du projet Eyeshome. Les aspects détaillés relatifs à sa conception, son implémentation, ses expérimentations et son évaluation sont présentés dans un rapport de projet distinct.

Le rapport est organisé en quatre grandes parties. La première présente le cadre général du stage et la structure d'accueil. La deuxième décrit les concepts techniques fondamentaux. La troisième présente le déroulement du stage ainsi que les principales activités réalisées, notamment l'étude du kit Keyestudio Arduino et la contribution au projet Eyeshome. Enfin, la quatrième partie présente le bilan du stage, les compétences acquises, les difficultés rencontrées ainsi que les perspectives qui en découlent.

Première partie

Cadre général du stage

Chapitre 1 Présentation de la structure d'accueil

Le Laboratoire IoT de l'École Nationale Supérieure Polytechnique de Yaoundé est un centre de recherche et de formation spécialisé dans l'Internet des Objets et les systèmes embarqués. Créé dans le cadre du projet Campus IoT, il dispose d'une infrastructure comprenant des microcontrôleurs variés (Arduino, STM32, ESP32), des cartes de développement spécialisées (Wyres Base, Nucleo), des gateways LoRaWAN, un serveur Chirpstack, ainsi qu'une bibliothèque complète de capteurs et de modules de communication. Le laboratoire poursuit plusieurs missions fondamentales : la recherche en IoT et en communication longue portée sans fil, la formation des étudiants aux technologies contemporaines, le prototypage et le test de solutions innovantes, ainsi que l'établissement de partenariats avec l'industrie et d'autres institutions académiques. Son environnement collaboratif et son infrastructure moderne en font un cadre propice à l'apprentissage pratique et à l'expérimentation de solutions IoT.

Chapitre 2 Organisation et déroulement du stage

Notre stage, d'une durée de près de sept semaines (du 8 juillet au 4 septembre 2026), a suivi une méthodologie progressive basée sur l'observation, l'apprentissage, la recherche, l'expérimentation et le travail en équipe. L'accueil initial a permis de nous familiariser avec les règles de fonctionnement du laboratoire et de présenter les premiers objectifs de travail. Les objectifs généraux incluaient l'acquisition de compétences en systèmes embarqués et en IoT, tandis que les objectifs spécifiques visaient à apprendre par la pratique, à mener des recherches approfondies, à expérimenter sur des cas concrets et à mettre en œuvre des solutions à grande échelle. La démarche adoptée a suivi le cycle suivant : observation initiale, apprentissage des concepts, recherche documentaire, expérimentation pratique, collaboration d'équipe, validation des résultats et amélioration continue, permettant une approche progressive et structurée de l'apprentissage.

Deuxième partie

Concepts techniques fondamentaux

Chapitre 3 Microcontrôleurs et systèmes embarqués

3.1 Qu'est-ce qu'un microcontrôleur ?

Un microcontrôleur est un circuit intégré complet qui rassemble, sur une seule puce, tous les éléments essentiels d'un mini-ordinateur : un processeur, de la mémoire et des interfaces d'entrée/sortie. Contrairement aux ordinateurs classiques, le microcontrôleur est spécialisé pour exécuter une ou plusieurs tâches précises de manière autonome, souvent en temps réel et avec des contraintes sévères en termes de consommation énergétique et d'espace.

Les microcontrôleurs se distinguent par leur spécialisation, leur autonomie fonctionnelle, l'intégration de tous les éléments sur une seule puce, leur efficacité énergétique remarquable et leur fiabilité de fonctionnement en continu. Ils sont omniprésents dans notre environnement quotidien : télécommandes, appareils électroménagers, électronique automobile (ABS, airbags), montres et trackers d'activité, accessoires IoT (capteurs, détecteurs).

Un microcontrôleur typique comprend un processeur capable d'exécuter des instructions, de la RAM pour les données temporaires, de la mémoire Flash pour le programme permanent, des interfaces de communication (UART, SPI, I2C), des ports GPIO pour le contrôle d'entrée/sortie, et une horloge interne pour synchroniser les opérations. Le fonctionnement suit le cycle fondamental : acquisition de données auprès des capteurs, traitement de ces informations selon un programme stocké en mémoire, et commande des actionneurs produisant une action adaptée.



FIGURE 3.1 – Microcontrôleur de la marque Microchip Technology

3.2 Fonctionnement général d'un microcontrôleur

Le fonctionnement d'un microcontrôleur peut se résumer en trois grandes étapes, qui se répètent en boucle tant que l'appareil est sous tension.

— **Acquisition** : le microcontrôleur reçoit des informations en provenance de capteurs connectés à ses broches d'entrée/sortie (appelées GPIO, pour *General Purpose Input/Output* — des broches configurables soit en entrée, soit en sortie selon le besoin). Ces informations peuvent être, par exemple, une température, un niveau de luminosité ou l'état d'un bouton.

— **Traitement** : le processeur du microcontrôleur interprète ces données selon les instructions du programme préalablement stocké dans sa mémoire. Ce traitement peut être une simple comparaison (« la température dépasse-t-elle un seuil ? ») ou un calcul plus complexe.

— **Action** : selon le résultat du traitement, le microcontrôleur commande un ou plusieurs actionneurs (allumer une LED, activer un relais, faire tourner un moteur) ou transmet les données traitées à un autre système, via une interface de communication (UART, SPI, I2C, Wi-Fi, etc.).

Ce cycle acquisition → traitement → action est rendu possible par une horloge interne, qui rythme l'exécution des instructions à une fréquence donnée, et par une mémoire divisée en plusieurs types : la mémoire Flash (qui conserve le programme même hors tension) et la RAM (qui stocke temporairement les données pendant l'exécution).

3.3 Architecture d'un microcontrôleur

L'architecture d'un microcontrôleur repose sur plusieurs composants interconnectés.

3.3.1 Unité centrale de traitement (CPU)

L'unité centrale de traitement (CPU) constitue le cœur du microcontrôleur. Elle interprète et exécute les instructions du programme stocké en mémoire, réalise les opérations arithmétiques et logiques, et contrôle le flux d'exécution des instructions. Elle comprend également des registres internes, qui constituent une mémoire ultra-rapide utilisée pour stocker temporairement les données nécessaires au traitement.

3.3.2 Mémoire

La mémoire d'un microcontrôleur peut prendre plusieurs formes :

— **RAM (Random Access Memory)** : mémoire volatile utilisée pour stocker temporairement les variables et les données pendant l'exécution du programme ;
— **ROM (Read-Only Memory)** : mémoire non volatile destinée à conserver des données ou des programmes qui ne sont généralement pas modifiés pendant le fonctionnement du microcontrôleur ;
— **Mémoire Flash** : mémoire non volatile et réinscriptible permettant notamment de stocker le programme principal de l'utilisateur. Elle facilite ainsi la reprogrammation du microcontrôleur.

3.3.3 Unités d'entrée/sortie (GPIO)

Les unités d'entrée/sortie, également appelées **GPIO (General Purpose Input/Output)**, constituent l'interface entre le microcontrôleur et son environnement extérieur. Elles permettent de recevoir des informations provenant de capteurs ou de boutons, mais également de commander des actionneurs tels que des moteurs, des relais ou des LED.

3.3.4 Interfaces de communication

Les microcontrôleurs disposent de différentes interfaces de communication permettant d'échanger des données avec des périphériques externes :

- **UART (Universal Asynchronous Receiver-Transmitter)** : utilisée pour les communications série simples ;
- **SPI (Serial Peripheral Interface)** : permet des échanges de données rapides avec des périphériques externes ;
- **I2C (Inter-Integrated Circuit)** : permet de connecter plusieurs périphériques en utilisant seulement deux lignes de communication.

Certains microcontrôleurs plus avancés intègrent également d'autres interfaces et technologies de communication telles que le **CAN (Controller Area Network)**, l'**USB**, l'**Ethernet**, le **Bluetooth** ou le **Wi-Fi**.

3.3.5 Horloge

L'horloge interne fournit le signal qui rythme l'exécution des instructions du microcontrôleur. Sa fréquence détermine notamment la vitesse à laquelle le microcontrôleur peut traiter les instructions. Certains microcontrôleurs disposent également d'horloges secondaires destinées à des modules spécifiques, notamment les modules temps réel ou les interfaces de communication nécessitant une synchronisation précise.

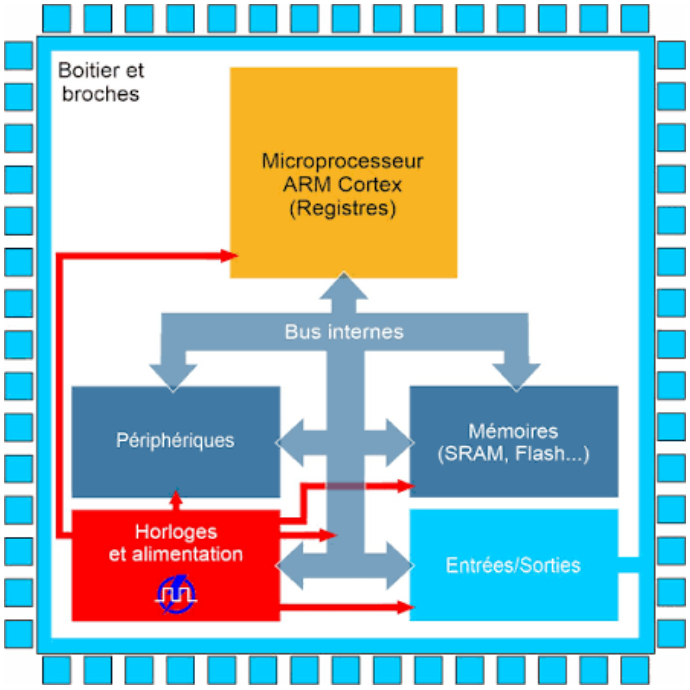


FIGURE 3.2 – Schéma fonctionnel d'un microcontrôleur

3.4 Exemples de microcontrôleurs courants

Il existe une grande variété de microcontrôleurs sur le marché, adaptés à différents besoins en termes de puissance de calcul, de consommation énergétique et de connectivité. Voici quelques exemples représentatifs :

— **Nordic nRF52 (Nordic Semiconductor)** : microcontrôleurs ARM Cortex-M4 spécialisés dans le Bluetooth Low Energy (BLE). Très utilisés dans les objets connectés portables (montres, bracelets, capteurs médicaux sans fil) grâce à leur très faible consommation en veille ;
— **Raspberry Pi RP2040 (Raspberry Pi Foundation)** : microcontrôleur double cœur ARM Cortex-M0+, apparu plus récemment sur le marché du prototypage (carte Raspberry Pi Pico). Il se distingue par son excellent rapport performance/prix et sa programmation flexible (C/C++, MicroPython) ;
— **Microcontrôleurs RISC-V (par exemple ESP32-C3 d'Espressif, ou GD32VF103 de GigaDevice)** : basés sur le jeu d'instructions ouvert RISC-V, en pleine expansion comme alternative libre aux architectures propriétaires ARM. L'ESP32-C3 combine cette architecture avec du Wi-Fi/Bluetooth intégré ;
— **Renesas RX / RA (Renesas Electronics)** : très présents dans l'automobile et l'industrie, réputés pour leur fiabilité et leurs fonctions de sécurité intégrées ;
— **Infineon XMC / PSoC (Infineon Technologies)** : orientés applications industrielles et automobiles, certains modèles PSoC intègrent des blocs analogiques programmables en plus du cœur numérique ;
— **STM8 (STMicroelectronics)** : microcontrôleurs 8 bits à bas coût, souvent utilisés dans l'électroménager et les applications simples ne nécessitant pas la puissance d'un cœur 32 bits.

Ces exemples montrent que le choix d'un microcontrôleur dépend toujours du contexte d'utilisation. Voici un tableau qui illustre quelques microcontrôleurs courants, leurs caractéristiques principales, leurs applications courantes et leurs points forts.

Microcontrôleur	**Caractéristiques principales**	**Applications courantes**	**Points forts**
nRF52 (Nordic)	ARM Cortex-M4, BLE intégré	Objets connectés portables	Très faible consommation en veille
RP2040 (Raspberry Pi)	Double cœur ARM Cortex-M0+	Prototypage, projets makers	Rapport performance/prix, flexibilité
ESP32-C3 / GD32VF103 (RISC-V)	32 bits RISC-V, Wi-Fi/Bluetooth (ESP32-C3)	IoT, alternative ouverte à ARM	Architecture libre, faible coût
RX / RA (Renesas)	Gammes automobile et industrielle	Automobile, industrie	Fiabilité, sécurité intégrée
XMC / PSoC (Infineon)	Blocs analogiques programmables (PSoC)	Applications industrielles et automobiles	Flexibilité analogique/numérique
STM8 (STMicroelectronics)	8 bits, bas coût	Électroménager, applications simples	Faible coût, simplicité

Table 3.1 – Tableau comparatif des microcontrôleurs additionnels

3.5 Lien entre microcontrôleurs et l'IoT

L'Internet des Objets désigne l'ensemble des objets physiques capables de collecter, traiter et échanger des données via Internet ou d'autres réseaux de communication. Ces objets « connectés » sont présents dans de nombreux domaines : domotique, santé, agriculture, industrie, transport, environnement. Leur but

est d'apporter une valeur ajoutée en rendant accessibles, en temps réel, des informations issues du monde physique pour améliorer la prise de décision, l'automatisation et le confort.

Dans ce contexte, les microcontrôleurs occupent une place centrale. La majorité des objets connectés reposent sur ces composants pour collecter des données à partir de capteurs (température, humidité, pression, mouvement), effectuer un premier traitement local, puis transmettre l'information vers d'autres dispositifs ou plateformes distantes via des protocoles de communication adaptés (Wi-Fi, Bluetooth, Zigbee, LoRa).

Leur faible consommation énergétique, leur coût réduit et leur capacité à intégrer de multiples interfaces de communication en font des éléments idéaux pour l'IoT. Les appareils doivent souvent fonctionner de manière autonome pendant de longues périodes, parfois avec une simple batterie. Le microcontrôleur agit comme le cœur intelligent de l'objet connecté, interagissant avec l'environnement physique, analysant ou pré-traitant les données localement, pilotant les actionneurs, et assurant la connectivité en transmettant les données vers le cloud ou d'autres appareils.

Chapitre 4 RIOT OS : Système d'exploitation pour l'IoT

4.1 Introduction à RIOT OS

L'essor de l'Internet des Objets a créé un besoin croissant pour des systèmes d'exploitation adaptés aux microcontrôleurs. Ces dispositifs, caractérisés par leurs ressources limitées en mémoire et en puissance de calcul, nécessitent des solutions logicielles spécialisées. RIOT OS répond à cette problématique en proposant un système d'exploitation temps réel, léger et portable.
RIOT OS (Real-time IoT Operating System) a été initialement développé par l'Institut National de Recherche en Informatique et en Automatique (INRIA) en France, en collaboration avec la Freie Universität Berlin. Lancé en 2013, ce projet open source vise à fournir une plateforme unifiée pour le développement d'applications IoT sur diverses architectures matérielles.
RIOT OS est un système d'exploitation open source conçu spécifiquement pour les microcontrôleurs et l'Internet des Objets. Développé pour répondre aux contraintes des systèmes embarqués à faibles ressources, RIOT offre une plateforme unifiée supportant de nombreuses architectures matérielles tout en maintenant une empreinte mémoire minimale.

4.2 Caractéristiques principales

RIOT OS adopte une architecture modulaire qui permet aux développeurs d'inclure uniquement les composants nécessaires à leur application. Cette approche contribue à minimiser l'empreinte mémoire du système final. Les modules principaux incluent le noyau (Kernel) pour la gestion des tâches et l'ordonnancement, les pilotes (Drivers) pour l'interface avec le matériel, la pile réseau supportant les protocoles de communication, le système de fichiers pour la gestion du stockage persistant, et les fonctions de cryptographie pour la sécurité.

RIOT OS privilégie l'utilisation des langages C et C++, familiers aux développeurs de systèmes embarqués. Cette approche facilite la portabilité du code et l'intégration avec les bibliothèques existantes.

Le système supporte plus de 200 cartes de développement basées sur différentes architectures : ARM Cortex-M (STM32, nRF52, ESP32), AVR (Arduino), MIPS, RISC-V, et x86 (pour la simulation et les tests).

RIOT implémente un ordonnanceur préemptif avec priorités fixes, garantissant des temps de réponse déterministes. Le système supporte le multitâche préemptif, la gestion des priorités, la synchronisation inter-tâches (mutex, sémaphores), et les timers avec gestion temporelle précise.

4.3 Avantages de RIOT OS

RIOT OS peut fonctionner avec aussi peu que 2 Ko de RAM et 8 Ko de mémoire Flash, ce qui le rend adapté aux microcontrôleurs les plus contraints. Cela représente une empreinte mémoire drastiquement inférieure à celle d'autres systèmes d'exploitation embarqués.

Le système privilégie l'utilisation de standards ouverts : POSIX pour la compatibilité API, IEEE 802.15.4 pour les communications radio, IPv6 natif avec 6LoWPAN, et CoAP pour les protocoles applicatifs. Ces

choix garantissent que les applications développées ne sont pas verrouillées à une plateforme spécifique.

RIOT offre un écosystème complet incluant un émulateur intégré pour les tests, le support GDB pour le débogage, des tests automatisés, une documentation extensive, et une communauté active de plus de 300 contributeurs.

4.4 Architecture technique de RIOT OS

RIOT utilise un modèle de tâches léger basé sur des threads coopératifs et préemptifs. Chaque thread possède sa propre pile et peut être dans l'un des états suivants : RUNNING (en cours d'exécution), PENDING (prêt à s'exécuter), SLEEPING (en attente d'un événement), ou BLOCKED (bloqué sur une ressource).

RIOT implémente plusieurs stratégies de gestion mémoire : allocation statique pour les ressources critiques, allocation dynamique optionnelle, et protection mémoire sur les architectures supportées.

Le système fournit plusieurs mécanismes de communication : messages asynchrones, mutex et sémaphores, files de messages (message queues), et mémoire partagée, permettant une coordination efficace entre les threads et les processus.

4.5 Installation et compilation

L'installation de RIOT OS sur un système Linux (Ubuntu) nécessite plusieurs étapes. L'installation des dépendances système inclut les outils de développement de base (git, build-essential, cmake), les compilateurs croisés (gcc-arm-none-eabi, gcc-avr), les outils de programmation et de débogage (openocd, gdb-arm-none-eabi, avrdude), Python et pip pour les outils supplémentaires, esptool pour ESP32/ESP8266, et minicom ou screen pour les terminaux série.

La compilation de RIOT pour différentes architectures suit un processus standard. Pour une compilation native sur Ubuntu (simulation), on utilise la commande : `make clean  make`. Pour ARM (STM32 Nucleo), la commande est : `make BOARD=nucleo-f401re clean  make BOARD=nucleo-f401re`. Pour ESP32, on utilise : `make BOARD=esp32-wroom-32 clean  make BOARD=esp32-wroom-32`. Pour Arduino, on utilise : `make BOARD=arduino-uno clean  make BOARD=arduino-uno`.

4.6 Applications et cas d'usage

RIOT OS est particulièrement adapté aux réseaux de capteurs sans fil grâce au support natif des protocoles 6LoWPAN et RPL, à la gestion avancée de l'énergie et aux pilotes pour de nombreux capteurs. Les applications domotiques bénéficient des protocoles de communication standardisés (CoAP, MQTT), de la sécurité intégrée (DTLS, AES), et de l'interopérabilité avec les clouds IoT. Dans le contexte industriel (Industrie 4.0), RIOT offre le déterminisme temps réel, la fiabilité et la robustesse, ainsi que les capacités de diagnostic et de maintenance prédictive.

Chapitre 5 La carte Wyres Base

5.1 Présentation générale

La carte Wyres Base est une plateforme électronique conçue pour le développement et le déploiement de solutions IoT basées sur la technologie LoRaWAN. Développée par l'entreprise française Wyres dans le cadre du projet Campus IoT, cette carte permet de connecter différents capteurs et dispositifs afin de collecter, traiter et transmettre des données sur de longues distances via un réseau LoRa.

Son rôle principal est de servir d'intermédiaire entre le monde physique et les systèmes numériques, en facilitant la communication entre capteurs et applications logicielles. Elle dispose d'un microcontrôleur basse consommation STM32L151CC de STMicroelectronics, d'un module de communication LoRa SX1272 de Semtech, ainsi que de plusieurs capteurs I2C tels qu'un accéléromètre, un baromètre et un microphone. Grâce à sa conception modulaire et à tous ces composants, la Wyres Base est adaptée à divers projets allant du prototypage à des applications industrielles ou environnementales.



FIGURE 5.1 – La carte Wyres Base

5.2 Fonctions principales

La carte Wyres Base offre plusieurs fonctions essentielles : la collecte de données permet à la carte de recevoir des informations provenant de divers capteurs (température, humidité, luminosité, mouvement, capteurs

environnementaux et industriels), mesurées en temps réel et préparées pour la transmission.

Le traitement local : grâce à son microcontrôleur intégré, la carte est capable de traiter et de filtrer les données directement sur la carte avant transmission, réduisant ainsi le volume de données envoyées et optimisant la consommation énergétique.

La communication longue portée via LoRaWAN : la carte dispose d'un module LoRaWAN assurant la transmission de données sur de longues distances, même dans des environnements difficiles, se connectant à une gateway LoRa qui relaie les informations vers un serveur ou une application cloud.

La compatibilité avec différents capteurs et modules : la Wyres Base est conçue pour être flexible et modulaire, permettant de connecter différents types de capteurs et périphériques selon les besoins du projet, incluant les connecteurs Grove pour simplifier le branchement de modules externes.

5.3 Caractéristiques techniques détaillées

Le microcontrôleur intégré est un STM32L151CC de STMicroelectronics disposant de 256 Ko de mémoire Flash, 32 Ko de RAM, 8 Ko d'EEPROM, 1 x UART, 2 x SPI, et 1 x I2C. C'est un microcontrôleur basse consommation capable de gérer la collecte, le traitement et la transmission des données.

Le module LoRaWAN assure la communication longue portée, fonctionnant généralement sur des bandes de fréquence adaptées aux réseaux IoT (868 MHz en Europe), permettant une transmission fiable sur plusieurs kilomètres, même dans des environnements urbains ou industriels denses.

Les connecteurs et capteurs incluent des connecteurs standardisés (notamment Grove) facilitant le branchement de capteurs externes sans soudure, supportant les communications UART et I2C. Les capteurs internes sont : l'accéléromètre ST LIS2DE12 (I2C), le baromètre ST LPS22HB (I2C), le microphone MEMS SPH0641LM4H-1 (I2S), et le capteur de luminosité sur GPIO.

L'alimentation et l'autonomie : la carte peut être alimentée par USB ou par batterie, offrant une grande flexibilité pour les projets mobiles ou distants. La faible consommation du microcontrôleur et du module LoRa contribue à une longue autonomie, essentielle pour les applications de terrain.

5.4 Connexion et programmation

Pour programmer la carte, il est nécessaire de repérer le port série auquel elle est connectée. Sur Linux, lister les ports avec : `dmesg | grep tty`. Le port est généralement `/dev/ttyUSB0`. Sur Windows, ouvrir le Gestionnaire de périphériques, section « Ports (COM et LPT) ». Sur macOS, utiliser : `ls /dev/tty.usb*`.

La programmation nécessite généralement un câble USB pour l'alimentation et la transmission, un module FTDI (convertisseur USB UART) si la carte n'a pas d'interface USB directe, et un programmateur ST-Link (v2, v3, ou intégré dans les cartes Discovery/Nucleo).

Les correspondances des broches entre le programmateur ST-Link et la Wyres Base sont : Pin 2 (VDD TARGET, Rouge) → Pin 3 (VDD), Pin 9 (SWCLK, Jaune) → Pin 4 (CLK), Pin 4 (GND, Noir) → Pin 2 (GND), Pin 7 (SWDIO, Bleu) → Pin 5 (DIO), Pin 15 (NRST, Vert) → Pin 1 (NRST).

5.4.1 Identification du port série et vérification de la connexion avec la carte Wyres Base

Avant de pouvoir programmer ou communiquer avec la carte Wyres Base, une étape préalable est indispensable : identifier le **port série** auquel elle est raccordée à l'ordinateur. Un port série est en quelque sorte l'« adresse » que l'ordinateur attribue à la carte une fois branchée par câble USB — sans cette adresse, aucun

logiciel ne peut savoir où envoyer ou recevoir les données échangées avec la carte. Cette identification dépend du système d'exploitation utilisé, ce qui nous a conduits à procéder différemment selon la machine employée :

— **Sous Windows**, le port apparaît dans le *Gestionnaire de périphériques*, dans la catégorie *Ports (COM et LPT)*, sous une forme telle que « USB Serial Device (COM3) ». Le numéro attribué (ici COM3) sert ensuite de référence pour tout logiciel de communication série ;
— **Sous Linux**, le port se retrouve généralement sous la forme `/dev/ttyUSB0` ou `/dev/ttyACM0`. La commande `dmesg | grep tty`, exécutée juste après le branchement de la carte, permet de repérer rapidement le périphérique nouvellement détecté par le système ;
— **Sous macOS**, le port se présente sous une forme proche de `/dev/tty.usbserial-XXXX`. La commande `ls /dev/tty.*` permet de lister l'ensemble des ports disponibles et d'identifier celui correspondant à la carte.

Une fois le port identifié, encore faut-il s'assurer que la communication fonctionne réellement. Pour cela, il suffit d'ouvrir un terminal série configuré sur ce port, avec un débit de communication (ou *baud rate*) fixé à 115200 bauds pour la carte Wyres Base. La commande à utiliser diffère elle aussi selon le système :

— **Sous Windows** : `mode COM3: BAUD=115200 PARITY=N DATA=8 STOP=1`
— **Sous Linux** : `screen /dev/ttyUSB0 115200`
— **Sous macOS** : `screen /dev/cu.usbserial-XXXX 115200`

Si des informations en provenance de la carte s'affichent alors dans le terminal, la connexion est confirmée et l'on peut poursuivre vers les étapes de programmation.

5.4.2 Branchement pour la programmation

La programmation de la carte Wyres Base nécessite deux connexions distinctes : l'une pour le transfert du programme vers la carte, l'autre pour l'alimentation et la communication série une fois le programme en place.

Pour l'alimentation et la transmission de données, un simple **câble USB** suffit. La carte Wyres Base ne disposant pas d'interface USB directe pour la communication série, un **module FTDI** (convertisseur USB UART) est nécessaire : il permet à l'ordinateur de dialoguer avec la carte comme s'il s'agissait d'un port série classique, ouvrant ainsi la voie au téléversement du code et à l'ouverture d'un terminal série.

Pour flasher le programme sur la carte, un **programmateur ST-Link** est indispensable. Dans notre cas, nous avons utilisé un ST-Link simple (autonome), sans passer par le programmateur intégré d'une carte Discovery ou Nucleo. Ce programmateur se connecte à la carte Wyres Base selon une correspondance de broches précise, résumée dans le tableau suivant :

ST-Link (JTAG ARM)	**Couleur du fil**	**Broche correspondante sur la Wyres Base**
Pin 2 : VDD_{TARGET}	Rouge	Pin 3 : VDD
Pin 9 : SWCLK	Jaune	Pin 4 : CLK
Pin 4 : GND	Noir	Pin 2 : GND
Pin 7 : SWDIO	Bleu	Pin 5 : DIO
Pin 15 : NRST	Vert	Pin 1 : NRST

TABLE 5.1 – Correspondance des broches entre le programmateur ST-Link et la carte Wyres Base

Il convient de noter que, sur les cartes Nucleo, les broches SWD sont directement étiquetées sur la carte elle-même, ce qui en facilite l'identification. Le programmateur ST-Link intégré à une carte Discovery ou Nucleo peut également, à titre indicatif, être utilisé pour charger un programme sur une autre carte, selon la correspondance de broches suivante :

ST-Link Nucleo	**Couleur du fil**	**Broche correspondante sur la Wyres Base**
SWD Pin 1 : VDDTARGET (VDD from application)	Rouge	Pin 3 : VDD
SWD Pin 2 : SWCLK (clock)	Jaune	Pin 4 : CLK
SWD Pin 3 : GND (ground)	Noir	Pin 2 : GND
SWD Pin 4 : SWDIO (SWD data input/output)	Bleu	Pin 5 : DIO
SWD Pin 5 : NRST (RESET of target STM32)	Vert	Pin 1 : NRST

TABLE 5.2 – Correspondance des broches entre ST-Link Nucleo et la carte Wyres Base

Une fois ces deux connexions établies — le câble USB/FTDI d'un côté, le ST-Link de l'autre — la carte est prête à recevoir le programme et à être supervisée via le terminal série.

5.5 Applications courantes de la carte Wyres Base

La carte Wyres Base est utilisée dans une grande variété de projets IoT, grâce à sa flexibilité et à ses capacités de communication longue portée. Parmi les applications les plus courantes, nous pouvons citer :

— **Géolocalisation indoor et outdoor** : la carte est spécialement conçue pour la localisation fine en intérieur. Elle permet de suivre des colis, des palettes ou des équipements avec une précision de 2 à 5 mètres, même dans de grandes surfaces comme des aéroports, des hôpitaux ou des entrepôts ;
— **Smart City** : surveillance de la qualité de l'air, gestion de l'éclairage public, suivi de la consommation énergétique, collecte de données environnementales pour améliorer la planification urbaine et le confort des habitants ;
— **Prototypage et recherche** : développement rapide de projets expérimentaux, démonstrateurs ou prototypes IoT, test et validation de capteurs et systèmes avant un déploiement à grande échelle.

La carte Wyres Base s'adapte à des contextes variés, qu'ils soient urbains, industriels, logistiques ou expérimentaux, offrant une solution fiable pour la collecte, le traitement et la transmission des données.

5.6 Avantages de la carte Wyres Base

La carte Wyres Base présente plusieurs atouts qui en font une plateforme privilégiée pour les projets IoT. Nous pouvons citer entre autres :

— **Précision de géolocalisation** : la carte intègre des algorithmes de localisation avancés permettant une précision de 2 à 5 mètres en intérieur, grâce à des protocoles comme TDoA (Time Difference of Arrival) et des technologies propriétaires développées par Wyres ;
— **Faible consommation énergétique** : le microcontrôleur STM32L151CC est optimisé pour un fonctionnement basse consommation, ce qui permet d'utiliser la carte sur batterie et d'assurer une autonomie prolongée pour les applications de terrain ;

— **Communication longue portée** : le module LoRa SX1272 intégré permet de transmettre des données sur plusieurs kilomètres, même dans des environnements urbains ou industriels denses, sans dépendre d'infrastructures réseau coûteuses ;
— **Capteurs embarqués** : la carte intègre plusieurs capteurs directement sur la carte (accéléromètre, altimètre, capteur de température, capteur de luminosité et microphone MEMS), ce qui réduit le besoin de composants externes ;
— **Flexibilité et compatibilité** : la carte supporte différents types de capteurs et périphériques via UART, I2C et SPI, ce qui facilite l'intégration de modules supplémentaires et l'adaptation aux besoins spécifiques d'un projet ;

La carte Wyres Base combine précision de localisation, autonomie, portée étendue et flexibilité, offrant ainsi une solution complète et pratique pour le développement et le déploiement de projets IoT.

Bien que la carte Wyres Base soit une solution flexible et pratique pour les projets IoT, elle présente des limitations liées à l'infrastructure nécessaire, à la programmation, à la portée, au nombre de périphériques et à la puissance de calcul, qui doivent être prises en compte dans la conception des projets.

5.7 Limites de la carte Wyres Base

Malgré ses nombreux avantages, la carte Wyres Base présente certaines limites qu'il est important de prendre en compte lors de son utilisation :

— **Dépendance à l'infrastructure LoRa** : pour transmettre les données vers un serveur ou une application cloud, la carte doit passer par une gateway LoRa. L'absence ou le mauvais positionnement de celle-ci peut limiter l'efficacité de la communication ;
— **Programmation nécessitant un outil spécifique** : contrairement aux cartes Arduino, la Wyres Base requiert un programmateur ST-Link v2 pour charger les programmes, ce qui peut être contraignant pour les débutants ;
— **Portée limitée par les obstacles** : bien que le module LoRa permette des communications longue portée, la présence de bâtiments, de murs ou d'environnements industriels denses peut réduire la distance effective de transmission ;
— **Nombre limité de périphériques I2C et UART** : la carte peut gérer plusieurs capteurs, mais le nombre de périphériques connectables simultanément reste limité, en particulier sur le bus I2C où l'adressage peut poser des contraintes ;
— **Puissance de calcul réduite** : le microcontrôleur STM32L151CC est adapté pour le traitement léger des données et le contrôle de capteurs, mais il n'est pas conçu pour des calculs complexes ou un traitement lourd en temps réel.

5.8 Schéma fonctionnel simplifié du fonctionnement de la carte Wyres Base

Pour mieux comprendre le rôle de la carte Wyres Base dans un projet IoT, il est utile de représenter le flux des données et les connexions principales.

— **Capteurs embarqués et externes** : la carte intègre directement un accéléromètre, un altimètre, un capteur de température, un capteur de luminosité et un microphone MEMS. Des capteurs externes peuvent également être connectés via les connecteurs Grove (UART, I2C) ;
— **Microcontrôleur STM32L151CC** : il collecte et traite les données reçues des capteurs, exécute le programme chargé dans sa mémoire (256 Ko Flash, 32 Ko RAM) et transmet les informations via le module LoRa ;

— **Module LoRa SX1272** : il assure la transmission des données vers une gateway LoRa via la technologie LoRaWAN, permettant une communication longue portée et basse consommation ;
— **Gateway LoRa** : elle relaie les données de la carte Wyres Base vers un serveur ou une application cloud ;
— **Serveur / Cloud** : il stocke, analyse et visualise les données. Il peut déclencher des alertes, des actions automatiques ou des mises à jour des systèmes connectés ;
— **Application utilisateur** : c'est l'interface permettant de consulter, analyser et exploiter les données collectées. Il peut s'agir d'un tableau de bord, d'une application mobile ou d'un logiciel de gestion.

Voici un schéma illustrant la chaîne complète de collecte et de transmission des données, tout en montrant les possibilités de programmation et de connexion avec la carte Wyres Base.

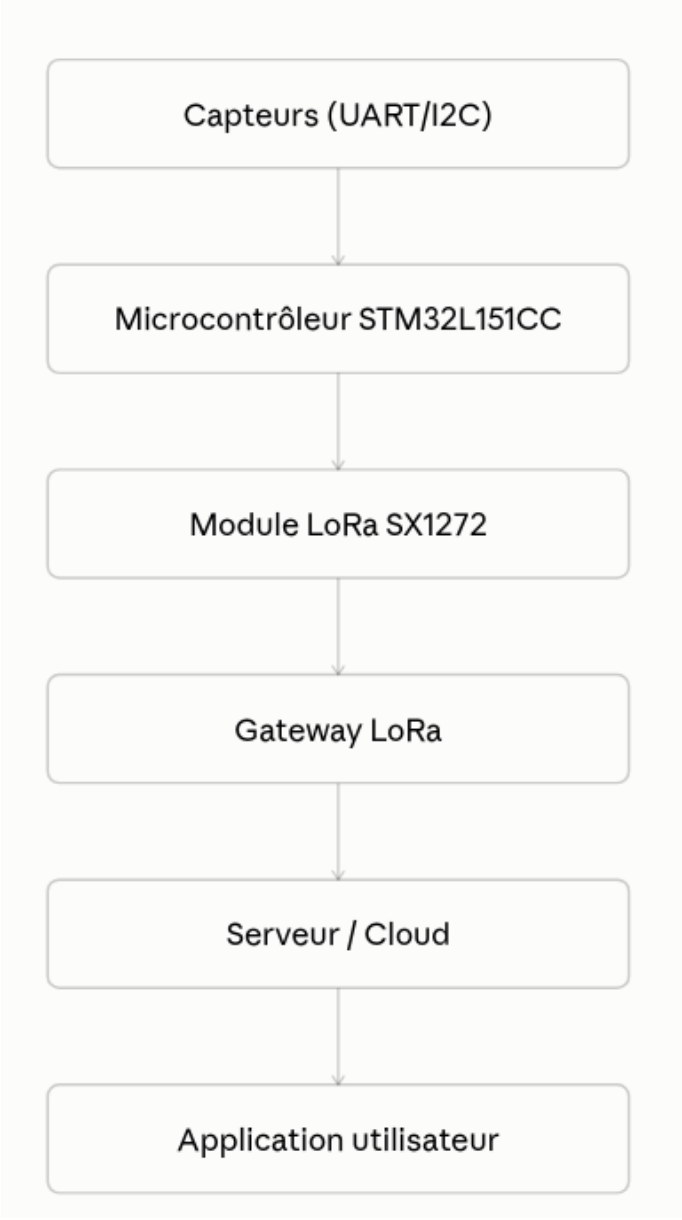


FIGURE 5.2 – Schéma fonctionnel simplifié de la carte Wyres Base dans un projet IoT

Chapitre 6 Technologies LoRa et LoRaWAN

6.1 LoRa : Technologie de modulation

LoRa (Long Range) est une technologie de modulation radio conçue pour les communications sans fil longue portée avec une très faible consommation d'énergie. LoRa permet aux appareils de transmettre des données sur de plus grandes distances que les technologies conventionnelles comme le Wi-Fi et le Bluetooth. Ce résultat est obtenu grâce à des techniques d'étalement de spectre, garantissant une transmission efficace même en présence d'interférences.

Les principales caractéristiques de LoRa sont : la longue portée, avec une transmission pouvant aller jusqu'à 20 km en zone rurale plane, variant entre 5 et 15 km en zone suburbaine et de 2 à 5 km en zone urbaine dense. La faible consommation d'énergie : les équipements LoRa consomment très peu d'énergie. Le faible coût de mise en œuvre : le matériel utilisé et les infrastructures mises en place sont très peu coûteux.

6.2 Comparaison avec d'autres technologies

Le tableau ci-dessous donne un schéma comparatif entre LoRa et d'autres technologies comme la 3G, la 4G, le Bluetooth, le Wi-Fi...

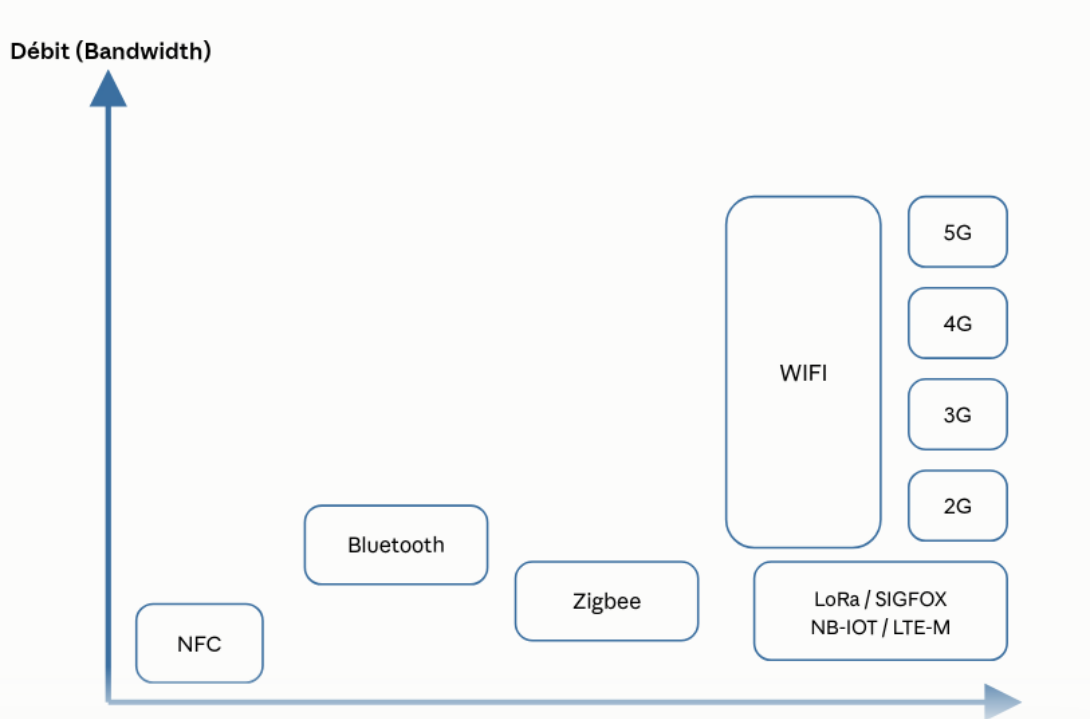


FIGURE 6.1 – Comparaison entre technologies

6.3 Quelques applications de LoRa

Grâce à sa faible consommation énergétique et à sa capacité à couvrir de longues distances, la technologie LoRa trouve des applications dans de nombreux domaines liés à l'Internet des Objets. Voici quelques exemples représentatifs :

— **Villes intelligentes (Smart City)** : suivi de la consommation énergétique, gestion de l'éclairage public, surveillance du niveau de remplissage des poubelles ou du trafic routier, dans le but d'optimiser la gestion des ressources urbaines ;
— **Agriculture connectée** : surveillance à distance de l'humidité du sol, de la température ou de l'ensoleillement des cultures, permettant une irrigation plus précise et une meilleure gestion des parcelles agricoles ;
— **Surveillance environnementale** : mesure de la qualité de l'air, de la température ou du niveau sonore dans des zones sensibles, utile pour la recherche scientifique ou la protection de l'environnement ;

6.4 LoRaWAN : Protocole réseau

Définition :

LoRaWAN (Long Range Wide Area Network) est un protocole LPWAN (Low Power, Wide Area Network), conçu pour connecter sans fil à l'Internet des Objets (IoT) des objets fonctionnant sur batterie, sur des réseaux régionaux, nationaux ou mondiaux.

6.5 Architecture LoRaWAN

Les principaux composants qui interviennent dans un réseau LoRaWAN sont les devices LoRa (capteurs de température, de pression, etc.), les gateways, le serveur d'application et le serveur de réseau, ainsi que l'application utilisateur.

Nous retrouvons d'un côté le device LoRa qui transmet une donnée. Cette donnée est réceptionnée à l'autre extrémité du réseau par un utilisateur. La structure globale du réseau LoRaWAN peut être représentée par la figure suivante :

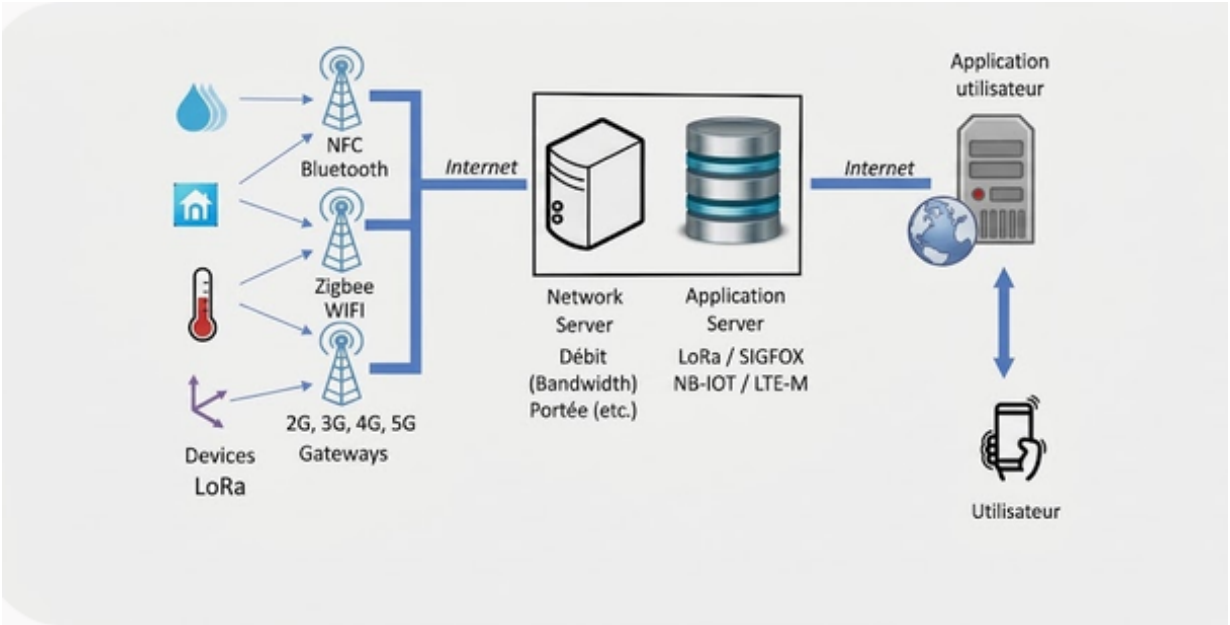


FIGURE 6.2 – Structure globale d'un réseau LoRaWAN

6.6 Classes de dispositifs LoRaWAN

Les devices LoRa sont classés en 3 catégories selon leur consommation et leur accessibilité en downlink :

Un device LoRa de classe A ne peut pas recevoir s'il n'a pas émis. Chaque émission (uplink) est suivie de 2 fenêtres de réception, durant lesquelles le device peut recevoir une information venant de la gateway. Il n'est donc pas joignable facilement. C'est la classe la plus économe en énergie.

Un device LoRa de classe B est joignable régulièrement sans qu'il soit nécessairement obligé d'émettre, car la gateway transmet des beacons au device, et après chaque beacon, deux fenêtres de réception se créent. Il consomme plus qu'un device de classe A mais moins qu'un device de classe C.

Un device LoRa de classe C est joignable en permanence, les fenêtres de réception étant toujours ouvertes. C'est la classe de device la plus énergétiquement exigeante.

6.7 Sécurité dans LoRaWAN

LoRaWAN définit deux couches de sécurité indépendantes : la sécurité du réseau (Network Security) garantit que l'appareil (end-device) est bien autorisé à utiliser le réseau, protégeant contre les intrusions non autorisées, basée sur une clé appelée NwkSKey (Network Session Key). La sécurité de l'application (Application Security) garantit la confidentialité des données échangées entre le capteur et le serveur applicatif, même l'opérateur du réseau ne peut pas lire le contenu du message, basée sur une clé appelée AppSKey (Application Session Key).

Deux modes sont possibles pour l'activation d'un device : OTAA (Over-The-Air Activation), recommandé, où le capteur contacte le réseau pour s'authentifier, avec deux clés uniques générées dynamiquement à partir d'une AppKey. C'est plus sécurisé car les clés changent à chaque session. ABP (Activation By Personalization), où les clés sont configurées en dur dans le capteur, moins sécurisé car il n'y a pas de renouvellement automatique.

Les mécanismes cryptographiques utilisent le chiffrement basé sur AES-128 (Advanced Encryption Standard, clé de 128 bits), l'utilisation de MIC (Message Integrity Code) pour garantir l'intégrité des messages, et la protection contre la modification des données, la falsification de messages, et la relecture (replay attacks) grâce à un compteur de trames (frame counter).

Les bonnes pratiques de sécurité incluent : toujours privilégier OTAA plutôt qu'ABP, utiliser des clés uniques par capteur, s'assurer que le compteur de trames est bien géré, stocker les clés de façon sécurisée (dans un module cryptographique), et mettre à jour régulièrement le firmware des capteurs.

Troisième partie

Déroulement et apprentissages du stage

Chapitre 7 Apprentissage pratique : Super Learning Kit for Arduino de Keyestudio (KS0077/KS0078/KS0079)

7.1 Présentation du kit et composants

Le Super Learning Kit for Arduino est une plateforme de développement éducative basée sur la carte Arduino Uno. Il permet de prendre en main les fondamentaux de l'électronique et de la programmation des microcontrôleurs de manière progressive. Le kit inclut une carte Arduino Uno (microcontrôleur ATmega328P), un vaste ensemble de capteurs (température, humidité, capteurs de distance, etc.), des actionneurs (servomoteurs, moteurs, LED RGB), des composants passifs (résistances, condensateurs), un breadboard pour les montages sans soudure, des câbles de connexion, et une documentation d'apprentissage progressive.



FIGURE 7.1 – Kit Keyestudio KS0077/KS0078/KS0079

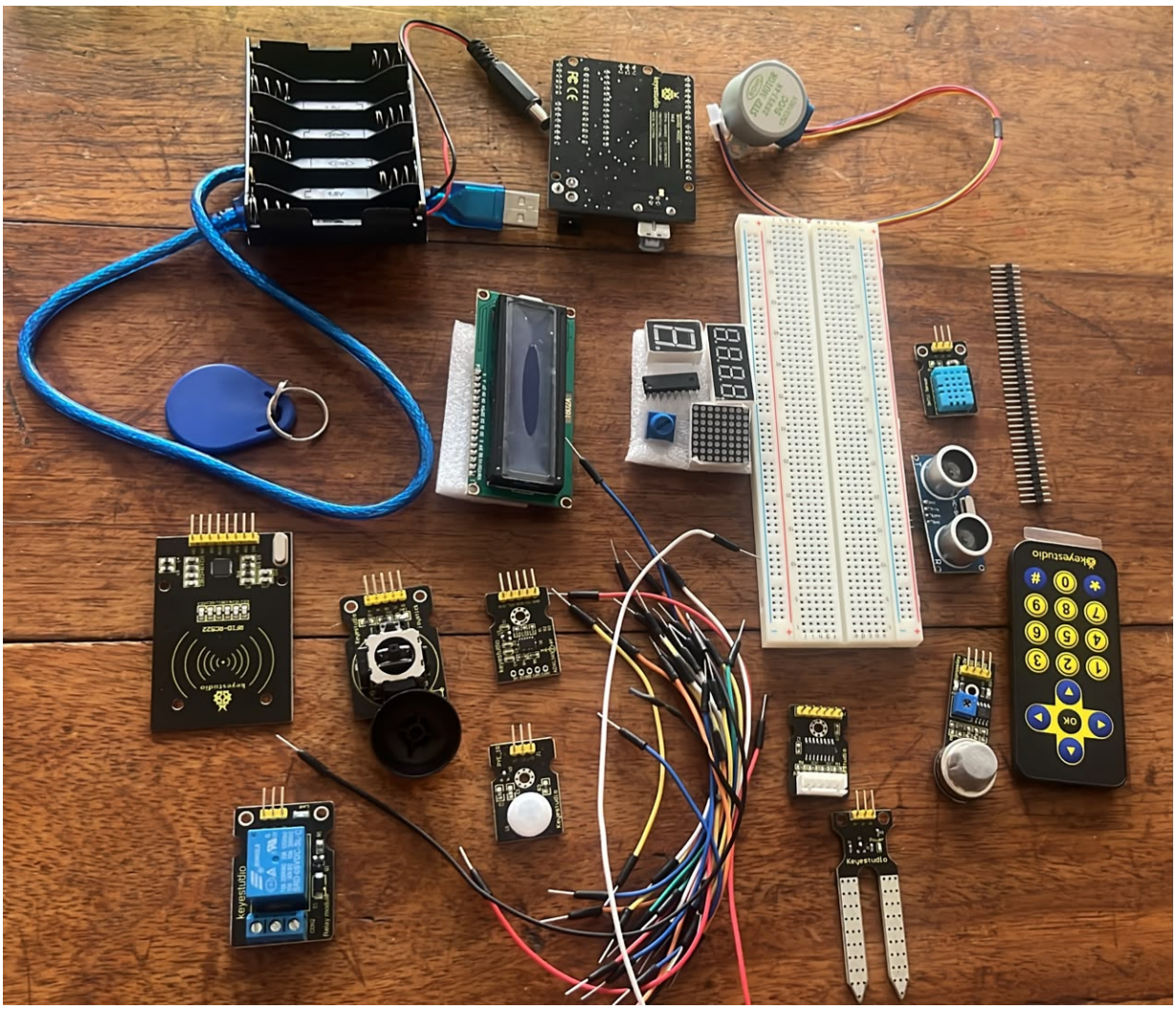


FIGURE 7.2 – Composants du kit Keyestudio KS0077/KS0078/KS0079

7.2 Étude des composants fondamentaux

La prise en main du kit a commencé par l'étude du microcontrôleur ATmega328P, comprenant son architecture 8 bits à 16 MHz, sa mémoire (32 Ko Flash, 2 Ko RAM, 1 Ko EEPROM), et ses interfaces (6 entrées analogiques, 14 broches numériques, UART, SPI, I2C).

Nous avons exploré les LED et la commande d'éclairage numérique, les boutons poussoirs et l'acquisition d'entrées numériques, les capteurs analogiques (température, luminosité, distance), les servomoteurs et moteurs pour l'actionnement, et les écrans LCD/OLED pour l'affichage des données.

7.3 Expérimentations réalisées

Les expérimentations ont suivi une progression de complexité croissante. La première série incluait des projets simples : clignotement de LED (Hello World embarqué), activation de LED par bouton, lecture et affichage de données de capteurs analogiques, commande de servomoteurs.

La deuxième série incluait des projets intégrés : système d'éclairage adaptatif basé sur la luminosité ambiante, thermostat simple avec capteur de température, système d'alerte avec seuils de détection, intégration de plusieurs capteurs simultanément.

À titre d'illustration, les images des projets 3 et 20 du kit sont présentées ci-dessous. L'ensemble des 32

projets est quant à lui consultable en détail via le lien du site du kit, fourni en annexe.

Le projet 3 du kit, consacré à la modulation de largeur d'impulsion (PWM), a permis de faire varier la luminosité d'une LED à l'aide d'un potentiomètre.

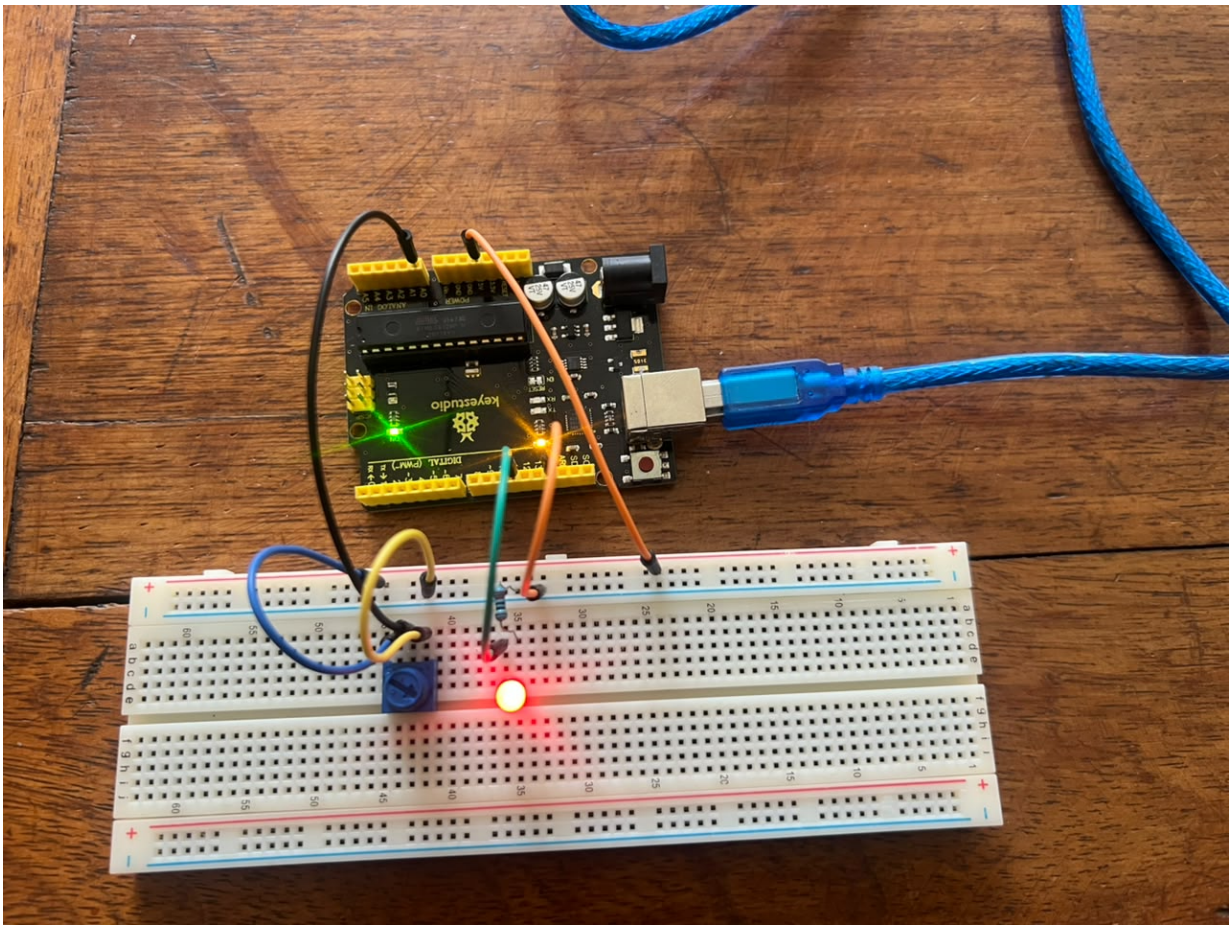


FIGURE 7.3 – Montage du projet PWM : variation de la luminosité d'une LED à l'aide d'un potentiomètre

Le projet 20 a consisté à afficher un message sur un écran LCD 1602, illustrant la communication avec un module d'affichage.

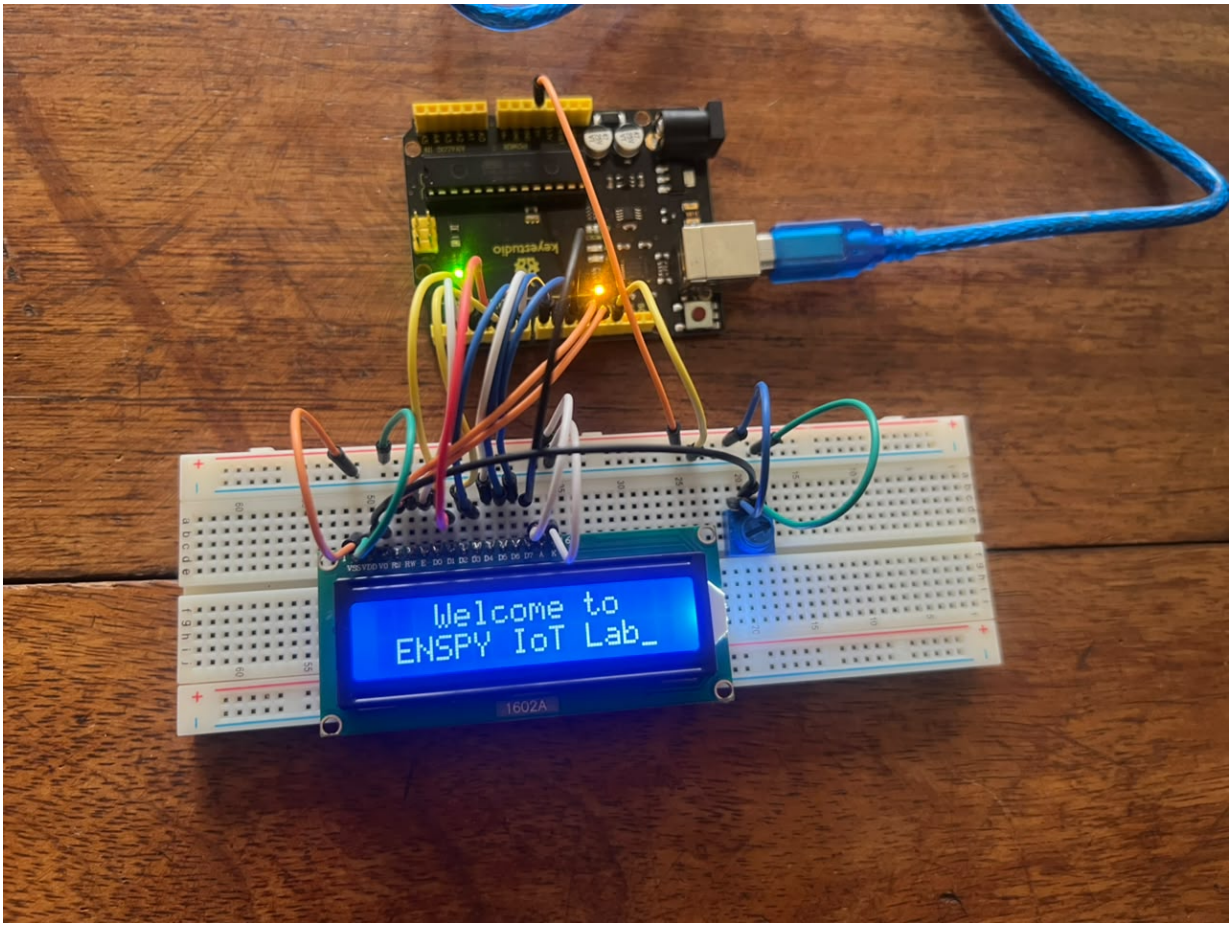


FIGURE 7.4 – Affichage d'un message sur l'écran LCD 1602

Chapitre 8 Participation au projet Eyeshome

8.1 Présentation du projet

Eyeshome est une solution orientée vers la surveillance intelligente d'un environnement domestique. Le projet vise à créer un système de monitoring complet permettant de suivre plusieurs paramètres d'une pièce ou d'un bâtiment en temps réel. Le système collecte des données sur la présence (détecteurs de mouvement), la température, l'humidité, et le niveau de gaz dangereux dans l'air, puis transmet ces informations en temps réel vers une application web (adaptée pour mobile) pour consultation et analyse.

L'objectif global est de créer une plateforme IoT complète démontrant l'intégration de capteurs, de microcontrôleurs, de protocoles de communication sans fil, et d'interfaces utilisateur pour la gestion domotique intelligente.

8.2 Organisation du travail en équipe

Le projet Eyeshome a été réalisé en équipe de cinq étudiants travaillant en collaboration étroite. L'organisation a suivi une structure claire avec une répartition des tâches basée sur les domaines de compétence : collecte et traitement des données, communication et réseau, interface utilisateur et application web, intégration matérielle, et documentation.

Les réunions d'équipe avaient lieu régulièrement pour discuter des avancées, résoudre les problèmes de synchronisation, et assurer la cohérence globale du système. La communication s'effectuait majoritairement via des canaux informels (discussions courantes au laboratoire).

Livrables et démarche du projet Eyeshome

Le projet **Eyeshome** a généré plusieurs livrables importants : un système matériel fonctionnel intégrant les capteurs et les microcontrôleurs, une application web permettant le monitoring en temps réel des paramètres, une documentation technique complète du système, ainsi qu'un rapport de projet détaillé décrivant la conception, l'implémentation et l'évaluation de la solution.

Phase 1 — Prototype basé sur la communication WiFi

La première étape du projet a consisté à établir un prototype fonctionnel reposant sur une communication **WiFi** entre la carte ESP32 et un script hébergé sur un serveur en ligne, permettant une première validation de la chaîne de collecte et de transmission des données.

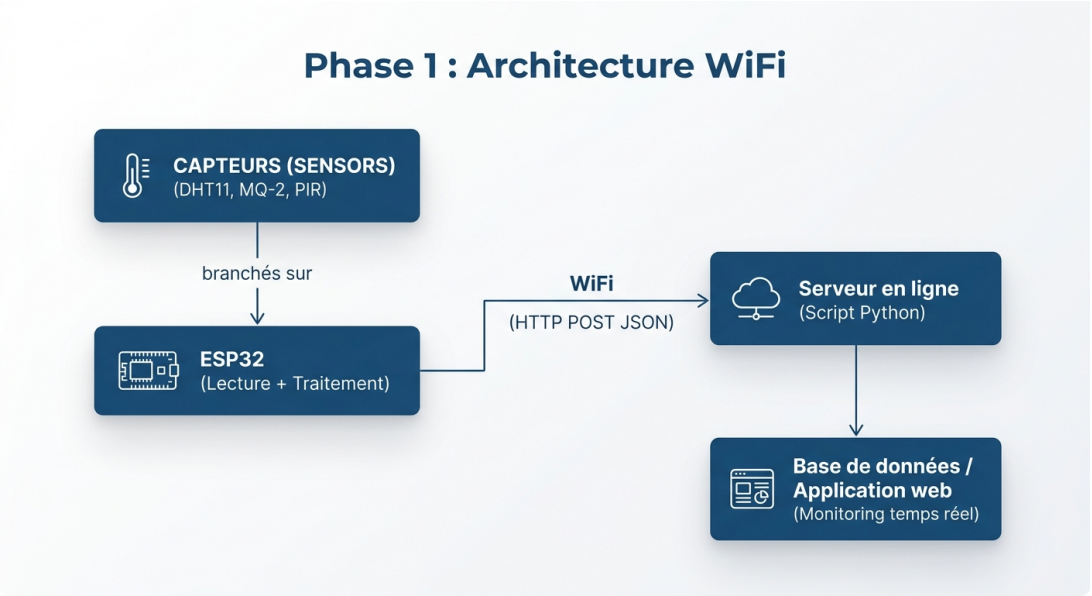


FIGURE 8.1 – Architecture suivie pour le prototype avec WiFi

Phase 2 — Migration vers la communication LoRa / LoRaWAN

La deuxième étape a consisté à faire évoluer l'architecture vers une communication **LoRa / LoRaWAN**, impliquant un émetteur LoRa et un récepteur LoRa (gateway), avec intégration de **ChirpStack** pour la gestion du réseau LoRaWAN et de **Node-RED** pour le traitement et l'acheminement des données.

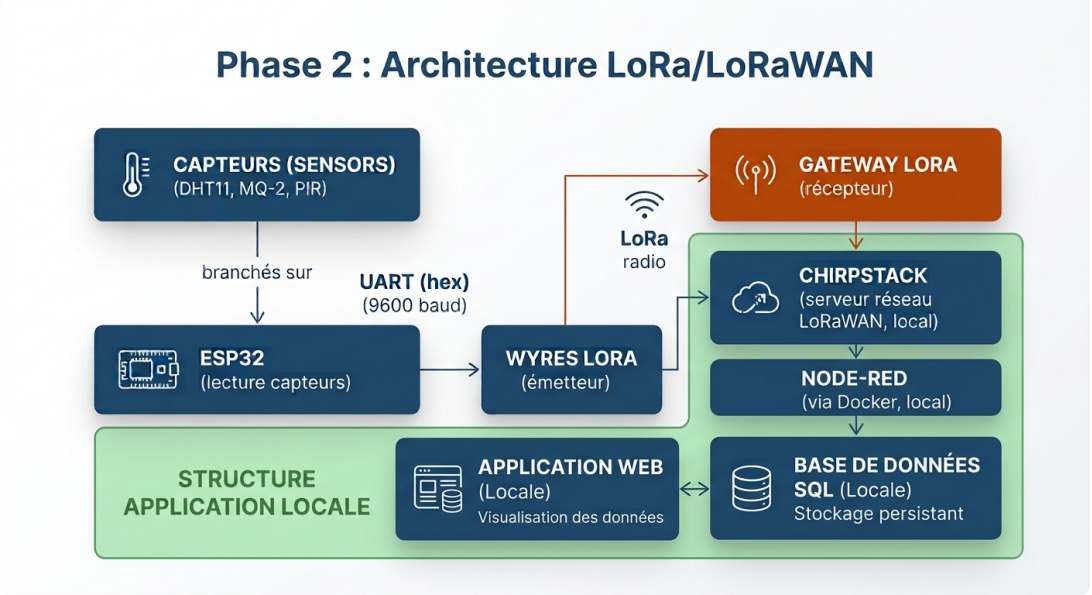


FIGURE 8.2 – Architecture suivie pour le prototype avec LoRa/LoRaWAN

Finalisation — Déploiement en local

Dans cette phase finale, l'ensemble de la solution a été consolidé pour fonctionner **entièrement en environnement local** : l'application de monitoring, le serveur de données, ainsi que **Node-RED**, désormais hébergé via **Docker**, ont été déployés localement, garantissant l'autonomie et l'indépendance du système vis-à-vis des services en ligne externes.

Le système a démontré avec succès sa capacité à collecter, traiter, transmettre et afficher des données IoT en temps réel, validant ainsi l'ensemble de la chaîne technologique étudiée durant le stage.

Quatrième partie

Bilan et perspectives

Chapitre 9 Compétences acquises

9.1 Compétences techniques

Les compétences techniques maîtrisées incluent la compréhension approfondie de l'architecture des microcontrôleurs, de la gestion mémoire, et des interfaces d'entrée/sortie. La programmation en C/C++ embarqué pour écrire du code efficace et optimisé pour des ressources limitées. L'utilisation de RIOT OS pour le développement multitâche sur systèmes embarqués, incluant la compilation multi-plateforme (ARM, AVR, ESP32).

L'étude des protocoles LoRa et LoRaWAN, incluant la configuration de devices et de gateways, la gestion de la sécurité réseau, et la communication longue portée. L'utilisation d'outils de développement incluant les environnements IDE, les compilateurs croisés, et les outils de débogage (OpenOCD). L'intégration de capteurs variés et la gestion des données acquises.

9.2 Compétences méthodologiques

Les compétences méthodologiques développées incluent l'analyse systématique de problèmes techniques complexes. La recherche documentaire efficace et la consultation de datasheets, documentations API, et tutoriels techniques. L'expérimentation progressive permettant de valider hypothèses et concepts théoriques. La documentation détaillée de processus et de solutions développées. Le débogage méthodique en utilisant des outils et techniques appropriés.

9.3 Compétences professionnelles

Les compétences professionnelles acquises incluent la communication technique claire et structurée auprès d'une équipe. Le travail collaboratif en équipe avec résolution constructive de conflits.

Le travail en équipe a apporté des bénéfices significatifs au-delà des compétences individuelles. Le partage de connaissances a permis une meilleure compréhension collective des problèmes complexes. La diversité des perspectives a conduit à des solutions plus robustes et innovantes. L'entraide mutuelle a accéléré la résolution de blocages techniques. Le respect des délais collectifs a développé le sens des responsabilités partagées. La dynamique de groupe a rendu le processus d'apprentissage plus enrichissant et motivant.

Chapitre 10 Difficultés rencontrées et solutions apportées

10.1 Difficultés techniques

Les difficultés techniques rencontrées incluaient des problèmes de compilation et de configuration des outils. La mise en place de l'environnement RIOT OS a d'abord présenté des défis : compilateurs croisés non correctement configurés, problèmes de PATH système, erreurs d'encodage (UTF-8.def introuvable). Les solutions apportées ont inclus l'installation correcte via gestionnaire de paquets, la vérification du PATH, et la suppression des packages problématiques (inputenc).

Les problèmes de communication série entre microcontrôleur et ordinateur se résolvaient généralement par ajustement des permissions (groupe dialout), réglage du taux de bauds correct (115200 pour RIOT), et test avec différents outils terminaux (majoritairement tio, parmi d'autres tels que minicom, screen, picocom).

10.2 Difficultés d'apprentissage

La courbe d'apprentissage abrupte lors de l'introduction à RIOT OS se justifiait par la complexité du système d'exploitation temps réel et la nécessité de comprendre le multitâche, la synchronisation et les interruptions. Les stratégies développées incluaient l'étude progressive des exemples RIOT, la consultation active de la documentation, et l'expérimentation itérative pour valider la compréhension.

La compréhension des protocoles LoRaWAN (sécurité, classes de devices, mécanismes d'activation) nécessitait une étude approfondie initialement. Les ressources disponibles, bien que nombreuses, étaient parfois fragmentées ou orientées commerce (documentation constructeur). La solution adoptée incluait la compilation de plusieurs sources documentaires, la création de notes de synthèse, et les discussions d'équipe avec les aînés pour clarifier les concepts.

10.3 Difficultés de travail en équipe

Au sein d'une équipe de cinq personnes, les difficultés initiales incluaient les différences de rythme d'apprentissage entre les membres, et la clarté des responsabilités. Les solutions développées incluaient la documentation claire des tâches assignées, et la communication proactive des blocages. Le temps a permis une meilleure synchronisation et une complémentarité accrue des compétences.

10.4 Enseignements tirés

Les enseignements principaux tirés de ces difficultés incluent l'importance cruciale de la persévérance face à une courbe d'apprentissage abrupte. La documentation minutieuse s'avère bénéfique pour les futurs projets et la communication en équipe. L'approche progressive et itérative est préférable à la tentative de comprendre tous les concepts simultanément. La communication proactive des difficultés accélère la résolution de problèmes. Le travail en équipe, malgré les défis initiaux, enrichit l'apprentissage et produit de meilleurs résultats.

Chapitre 11 Bilan critique et perspectives

11.1 Bilan global du stage

Ce stage de près de sept semaines au Laboratoire IoT de l'ENSPY a constitué une expérience d'apprentissage intense et enrichissante. Les trois axes du stage, notamment l'apprentissage théorique des concepts techniques, la pratique avec le kit Keyestudio Arduino, et la contribution active au projet Eyeshome, ont permis une intégration harmonieuse de la connaissance théorique et de l'expérience pratique.

Les objectifs pédagogiques ont été atteints : acquisition de compétences en systèmes embarqués, maîtrise de technologies IoT contemporaines, contribution à un projet collectif, et développement de compétences professionnelles transversales. Le stage a dépassé les attentes initiales en offrant une expérience intégrative de l'ensemble de la chaîne IoT.

11.2 Analyse critique

Les points positifs du stage incluent la pertinence des technologies apprises par rapport aux besoins réels du marché IoT, la qualité de l'infrastructure et des ressources disponibles au laboratoire, et l'encadrement professionnel et bienveillant fourni tout au long du stage. La diversité des activités (théorie, pratique, projet collectif) a permis un apprentissage riche et nuancé.

Les points à améliorer incluent une introduction plus progressive aux concepts complexes (RIOT OS aurait bénéficié de modules d'apprentissage structurés supplémentaires), une documentation en français plus abondante pour certains outils, et peut-être une meilleure intégration entre l'étude Keyestudio et le projet Eyeshome (les deux auraient pu être davantage liés).

11.3 Contribution au laboratoire

Notre réalisation au projet de surveillance d'une pièce par un dispositif (Eyeshome), grâce à la documentation produite, enrichit le laboratoire et lui apporte une valeur ajoutée. Notre retour d'expérience sur les processus d'apprentissage, les outils, et les méthodologies peut aider à améliorer les expériences futures des futurs stagiaires.

11.4 Perspectives et évolutions futures

Les perspectives d'approfondissement technique incluent l'exploration de domaines spécialisés comme la sécurité IoT avancée, le machine learning embarqué sur microcontrôleurs, ou la conception de protocoles de communication personnalisés. Sur le plan professionnel, les apprentissages acquis ouvrent des portes vers des carrières en ingénierie des systèmes embarqués, développement IoT, ou recherche en systèmes temps réel.

Une continuation logique serait la participation à des projets de recherche plus avancés, l'approfondissement de domaines spécifiques (sécurité, efficacité énergétique, scalabilité), ou la transition vers un rôle professionnel dans une entreprise développant des solutions IoT. Les apprentissages généraux, parmi lesquels la rigueur,

la persévérance, la curiosité et la capacité d'apprentissage autonome, seront précieux quelle que soit la trajectoire professionnelle ultérieure.

Conclusion générale

Ce stage a représenté bien plus qu'une simple accumulation de connaissances techniques. C'est une véritable transformation de notre compréhension du rôle crucial des systèmes embarqués dans notre monde de plus en plus connecté. Nous avons progressé de concepts fondamentaux (architecture des microcontrôleurs) vers des technologies avancées (LoRaWAN, RIOT OS), tout en mettant en pratique ces apprentissages dans un projet concret et collectif.

Les technologies maîtrisées telles que les microcontrôleurs, RIOT OS, les cartes Wyres Base, et les protocoles LoRa/LoRaWAN forment le fondement de l'infrastructure IoT moderne. Mais au-delà des aspects purement techniques, ce stage a développé des compétences professionnelles et personnelles d'égale importance : la capacité à apprendre de manière autonome, à collaborer efficacement dans une équipe, à communiquer des concepts techniques complexes, et à persévérer face aux défis.

Les défis rencontrés, courbe d'apprentissage abrupte, problèmes de compilation, difficultés d'intégration d'équipe, ne sont pas des échecs mais des opportunités d'apprentissage. Chacun a contribué à développer la résilience et la capacité de résolution de problèmes.

Au Laboratoire IoT de l'ENSPY, nous avons non seulement appris des technologies, nous avons appris à apprendre, à nous adapter, à contribuer à un collectif plus grand.

Nous remercions une fois de plus notre encadreur et l'équipe du laboratoire d'avoir rendu possible ce stage.

Références bibliographiques

Rapport de stage de 2024/2025

RIOT OS Official Website, https://riot-os.org/

LoRa Alliance, https://lora-alliance.org/

Wyres Technologies, Documentation de la carte Wyres Base

Arduino Official Documentation, https://docs.arduino.cc/

Keyestudio Documentation and Tutorials, https://keyestudio.com/

STMicroelectronics STM32L151CC Datasheet, https://www.st.com/

MQTT Protocol Specification, https://mqtt.org/

What is the Internet of Things (IoT) ?, https://www.iot.org/

Embedded Systems Design - ARM Architecture Reference